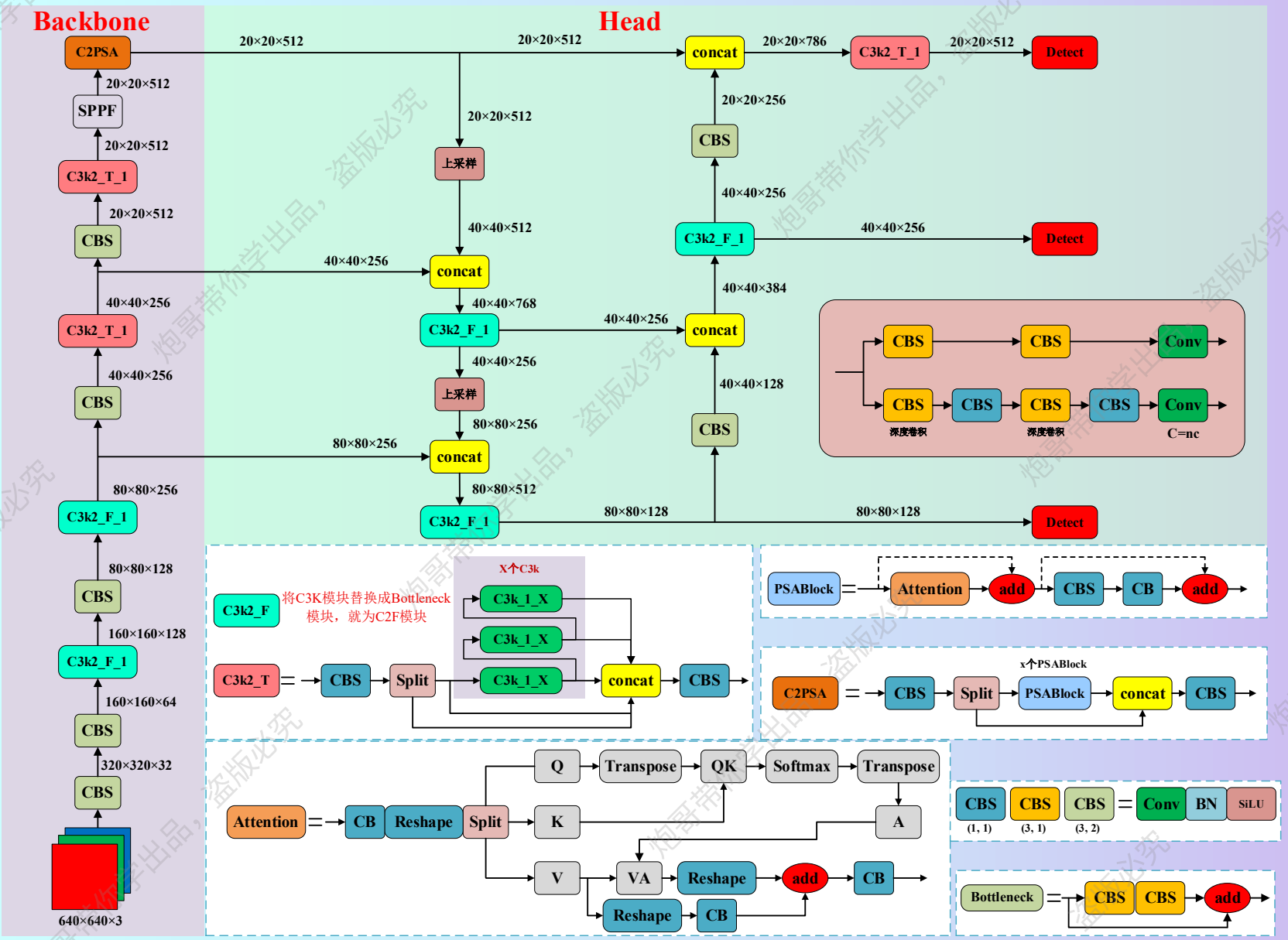
Yolov11 目标检测原理



Yolov11目标检测算法的背景

Yolov11（其实应该叫yolo11）是 Ultralytics公司开发于2024年9月30日发布，Yolov11是以源码的形式发布，没有发布相关论文。值得一提的是如果说yolov3奠定了yolo的架构， Ultralytics发布的yolov5、yolov8、yolov11奠定了代码的架构，使得yolo系列发扬光大。Yolov11是所有版本的集大成之作，主要工作如下：

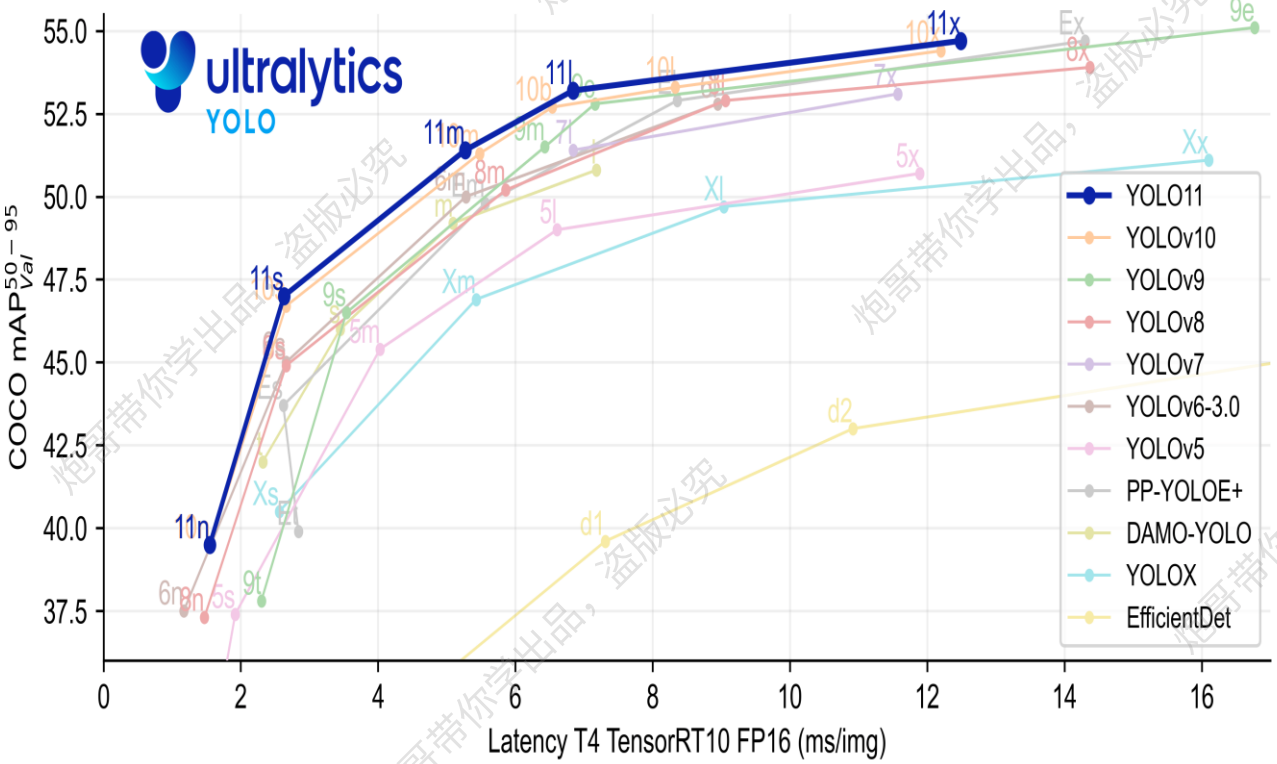
C3k2 模块：借鉴了v5中的C3模块和v8中的c2f模块

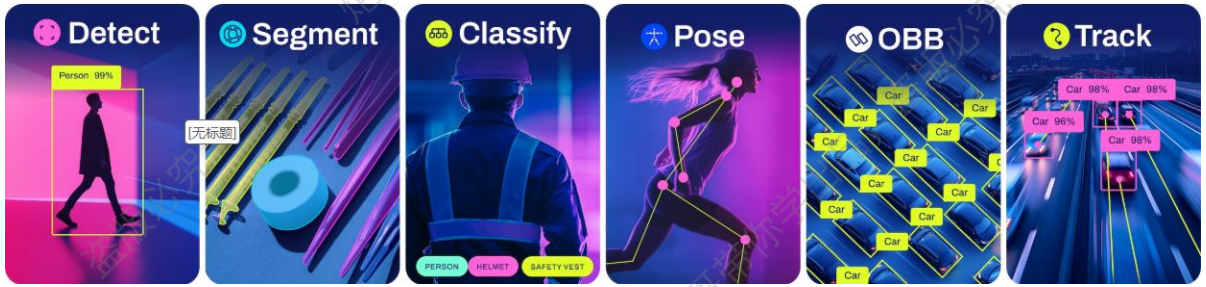
C2PSA模块：借鉴了v10中的PSA模块并进行优化

检测头：利用深度可分离卷积，完整借鉴v10检测头

精度和速度的均衡：更高的精度和更少参数的模型的均衡。

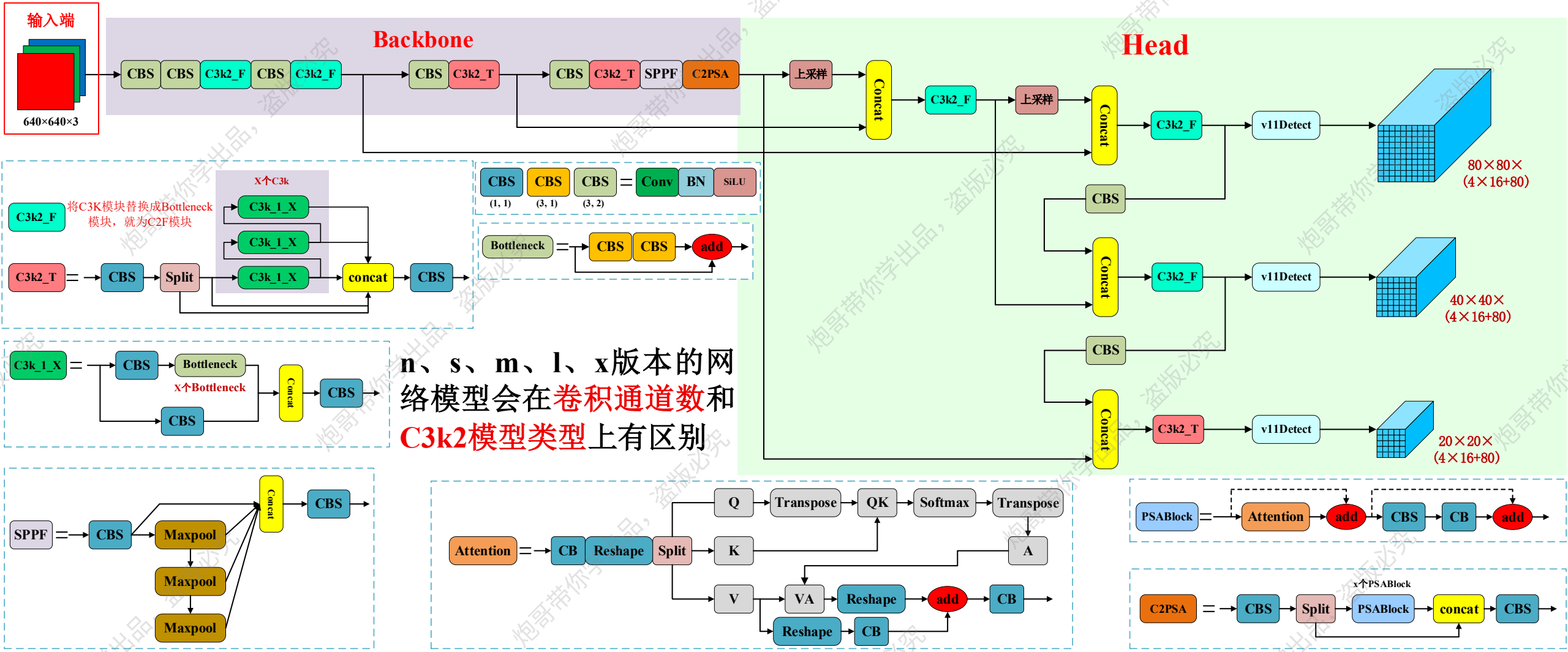
多任务学习：支持目标检测、实例分割、姿态估计等多任务。





炮哥带你学出品，盗版必究

Yolov11整体网络结构



Yolov11不同模型结构模型参数

```
# [depth, width, max_channels]
n: [0.50, 0.25, 1024] # summary:
s: [0.50, 0.50, 1024] # summary:
m: [0.50, 1.00, 512] # summary:
l: [1.00, 1.00, 512] # summary:
x: [1.00, 1.50, 512] # summary:
```

depth：C3K2和C2PSA中模块重复次数

Width：整体网络模型通道数缩放比例

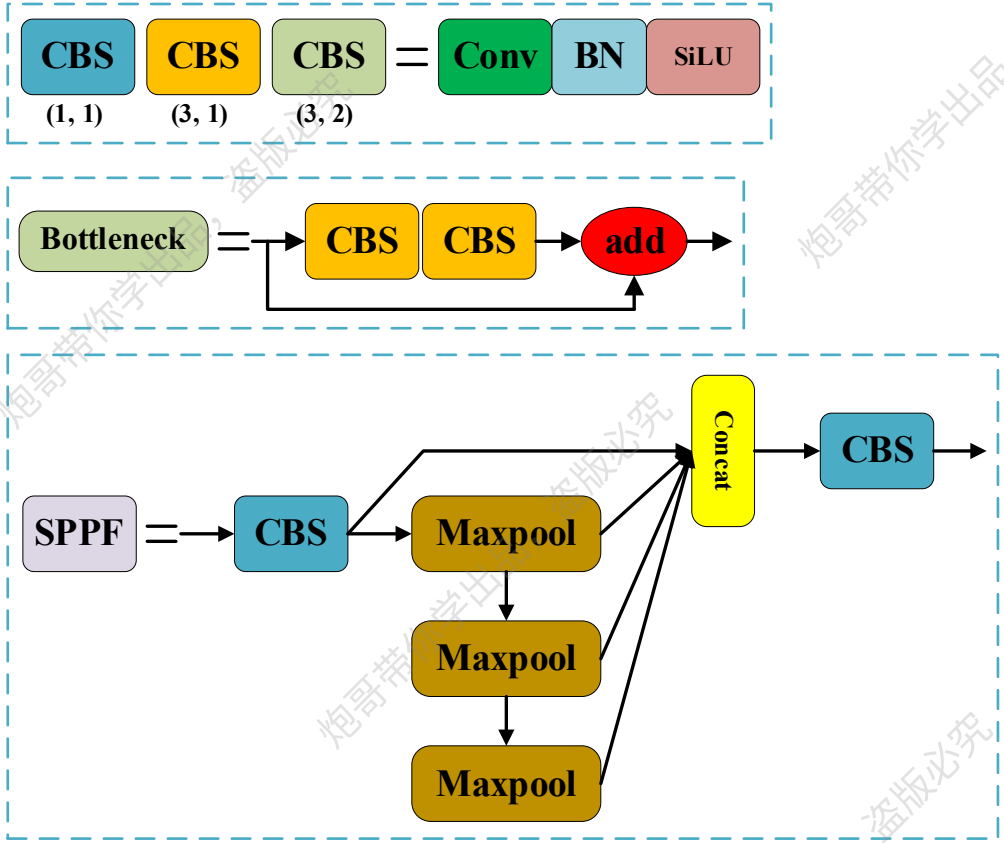
最大通道数大小：width×maxchannels

注意：m、l、x号模型中的C3K2模块的参数都为True

```
       from  n    params  module                                       arguments
         -1  1       928  ultralytics.nn.modules.conv.Conv             [3, 32, 3, 2]
         -1  1     18560  ultralytics.nn.modules.conv.Conv             [32, 64, 3, 2]
         -1  1     26080  ultralytics.nn.modules.block.C3k2            [64, 128, 1, False, 0.25]
         -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
         -1  1    103360  ultralytics.nn.modules.block.C3k2            [128, 256, 1, False, 0.25]
         -1  1    590336  ultralytics.nn.modules.conv.Conv             [256, 256, 3, 2]
         -1  1    346112  ultralytics.nn.modules.block.C3k2            [256, 256, 1, True]
         -1  1   1180672  ultralytics.nn.modules.conv.Conv             [256, 512, 3, 2]
         -1  1   1380352  ultralytics.nn.modules.block.C3k2            [512, 512, 1, True]
         -1  1    656896  ultralytics.nn.modules.block.SPPF            [512, 512, 5]
         -1  1    990976  ultralytics.nn.modules.block.C2PSA           [512, 512, 1]
         -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
    [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
         -1  1    443776  ultralytics.nn.modules.block.C3k2            [768, 256, 1, False]
         -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
    [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
         -1  1    127680  ultralytics.nn.modules.block.C3k2            [512, 128, 1, False]
         -1  1    147712  ultralytics.nn.modules.conv.Conv             [128, 128, 3, 2]
   [-1, 13]  1         0  ultralytics.nn.modules.conv.Concat           [1]
         -1  1    345472  ultralytics.nn.modules.block.C3k2            [384, 256, 1, False]
         -1  1    590336  ultralytics.nn.modules.conv.Conv             [256, 256, 3, 2]
   [-1, 10]  1         0  ultralytics.nn.modules.conv.Concat           [1]
         -1  1   1511424  ultralytics.nn.modules.block.C3k2            [768, 512, 1, True]
[16, 19, 22]  1    827148  ultralytics.nn.modules.head.Detect           [20, [128, 256, 512]]
mary: 181 layers, 9,435,532 parameters, 9,435,516 gradients, 21.6 GFLOPs
```

炮哥带你学出品，盗版必究

Yolo中的一直沿用的基本模块



如果你对各个版本的yolo网络模型很熟悉的话，你会发现上述的几个模块在v5后就一直存在，同时右边的用yaml文件的形式来定义网络结构一直延续下来。因此不得不感谢Ultralytics（v5，v8，v11）将yolo系列真正发扬光大和落地应用。

```
# Parameters
nc: 80 # number of classes
scales: # model compound scaling constants, i.e. 'model=yolo11n.y
  # [depth, width, max_channels]
  n: [0.50, 0.25, 1024] # summary: 181 layers, 2624080 parameters
  s: [0.50, 0.50, 1024] # summary: 181 layers, 9458752 parameters
  m: [0.50, 1.00, 512] # summary: 231 layers, 20114688 parameters
  l: [1.00, 1.00, 512] # summary: 357 layers, 25372160 parameters
  x: [1.00, 1.50, 512] # summary: 357 layers, 56966176 parameters

# YOLO11n backbone
backbone:
  # [from, repeats, module, args]
  - [-1, 1, Conv, [64, 3, 2]] # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]] # 1-P2/4
  - [-1, 2, C3k2, [256, False, 0.25]]
  - [-1, 1, Conv, [256, 3, 2]] # 3-P3/8
  - [-1, 2, C3k2, [512, False, 0.25]]
  - [-1, 1, Conv, [512, 3, 2]] # 5-P4/16
  - [-1, 2, C3k2, [512, True]]
  - [-1, 1, Conv, [1024, 3, 2]] # 7-P5/32
  - [-1, 2, C3k2, [1024, True]]
  - [-1, 1, SPPF, [1024, 5]] # 9
  - [-1, 2, C2PSA, [1024]] # 10
```

不同版本的深度
宽度和最大通道数
参数

该层输入来源

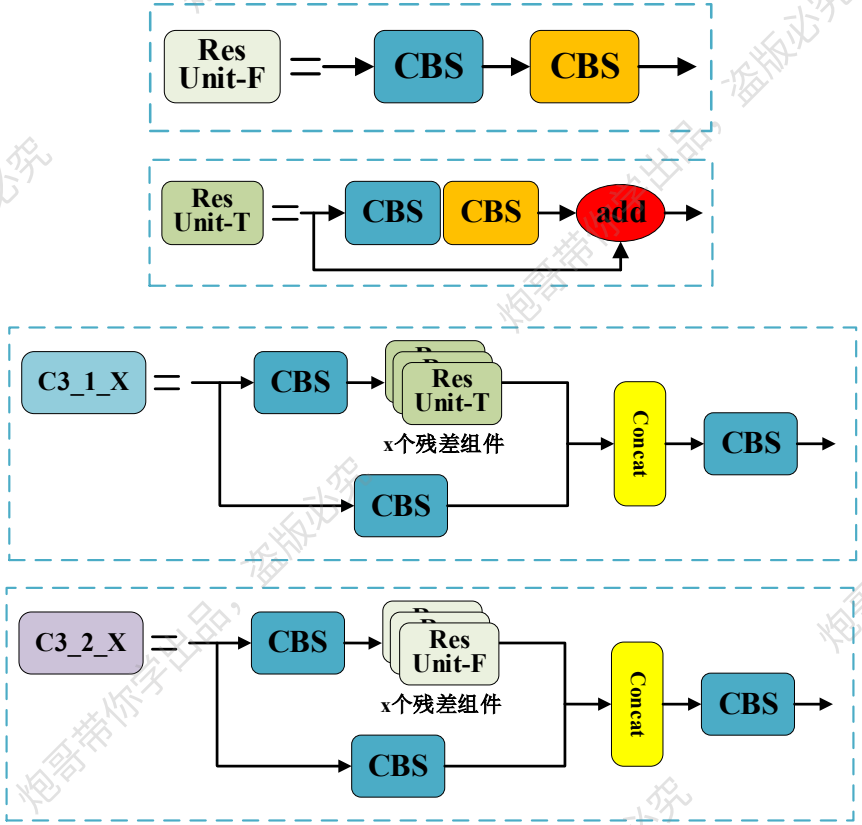
该层模块重复次数和模块名称

模块参数

炮哥带你学出品，盗版必究

Yolov11中的C3k2模块

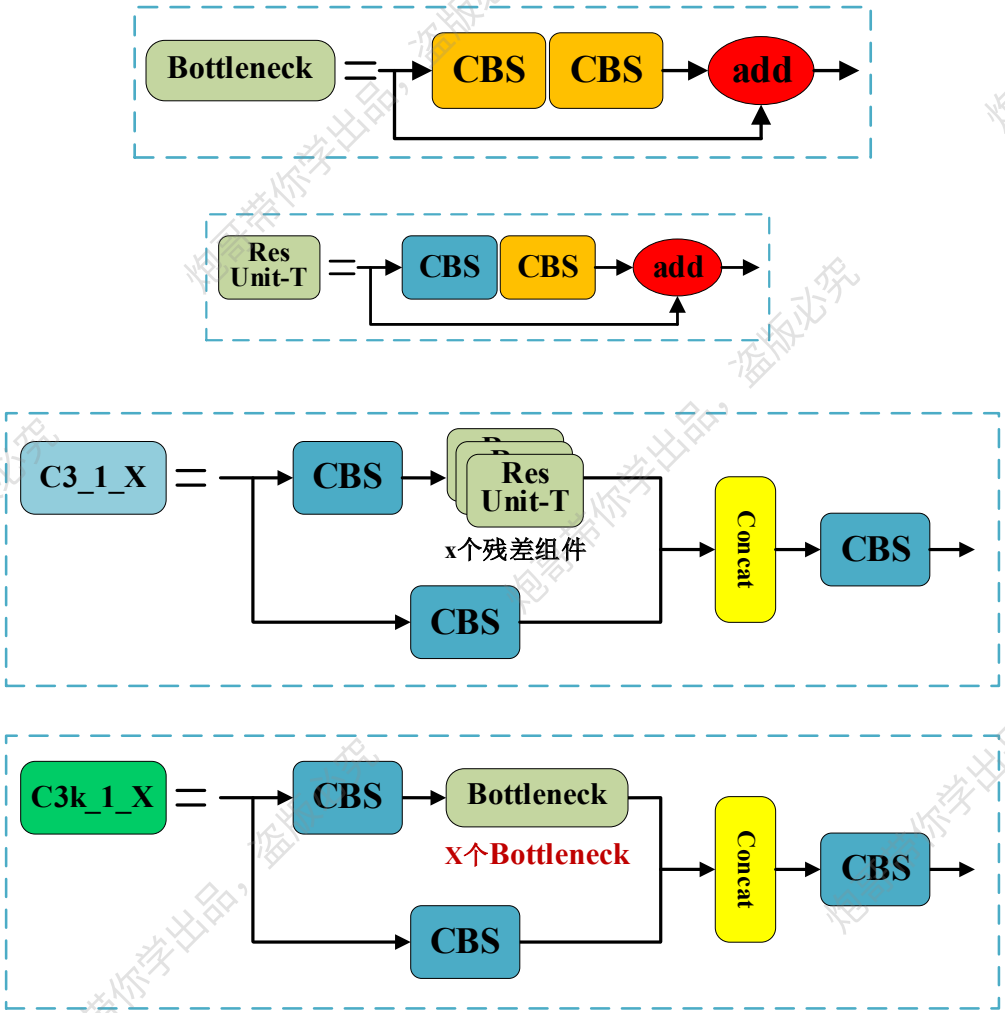
前面有提到过Yolov11集成了前面不同版本的优势进行模型构建，主要参考版本有yolov5、yolov7、yolov8、yolov10。因此在讲解yolov11的主要模块的时候需要回顾前面的模块。




```python
class C3(nn.Module):  11 usages
    def __init__(self, c1: int, c2: int, n: int = 1, shortcut: bool = True,
                 g: int = 1, e: float = 0.5):
        """
        参数:
            c1 (int): 输入通道数。
            c2 (int): 输出通道数。
            n (int): Bottleneck模块的数量。
            shortcut (bool): 是否使用shortcut连接（跳跃连接）。
            g (int): 卷积操作中的分组数。
            e (float): 扩展比例，决定隐藏通道的数量。
        """
        super().__init__()
        c_ = int(c2 * e)  # 隐藏通道数
        # 初始化3个卷积层，分别处理输入数据
        self.cv1 = Conv(c1, c_, k: 1, s: 1)  # 第一层卷积，1x1卷积
        self.cv2 = Conv(c1, c_, k: 1, s: 1)  # 第二层卷积，1x1卷积
        self.cv3 = Conv(2 * c_, c2, k: 1)  # 第三层卷积，1x1卷积，连接后的结果会通过该卷积层输出
        # 使用多个Bottleneck模块作为CSP Bottleneck的一部分
        self.m = nn.Sequential(*(Bottleneck(c_, c_, shortcut, g, k=((1, 1), (3, 3)), e=1.0)
                                 for _ in range(n)))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # 将cv1和cv2的输出通过m模块处理后合并，最后通过cv3卷积输出
        return self.cv3(torch.cat( tensors: (self.m(self.cv1(x)), self.cv2(x)), dim: 1))
```

炮哥带你学出品，盗版必究

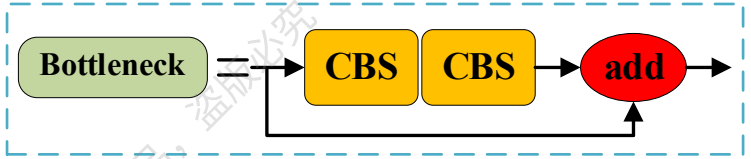
Yolov11中的C3k2模块

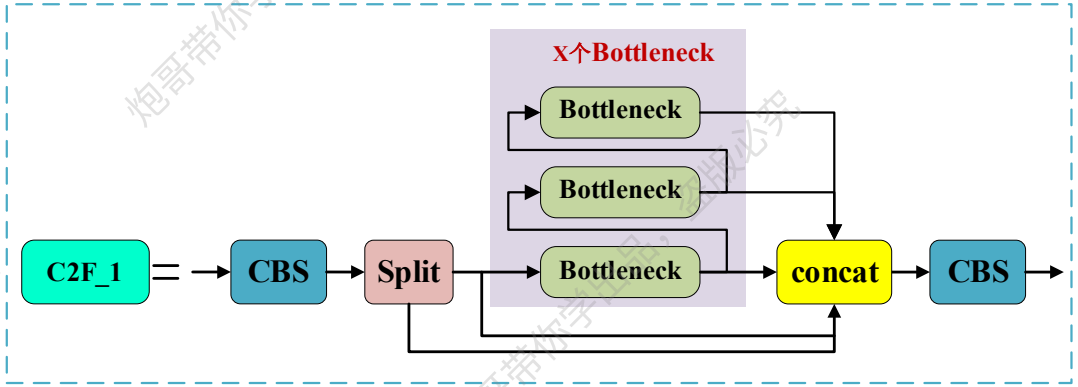


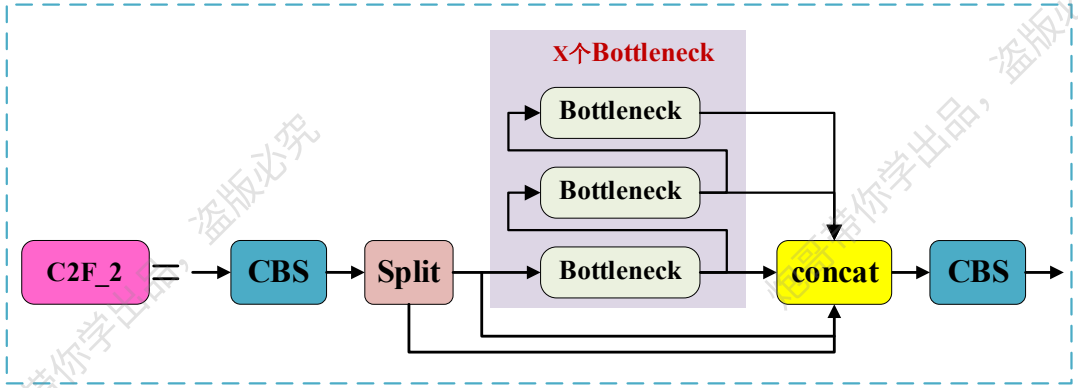

```python
class C3k(C3):  2 usages
    def __init__(self, c1: int, c2: int, n: int = 1, shortcut: bool = True,
                 g: int = 1, e: float = 0.5, k: int = 3):
        """
        参数:
            c1 (int): 输入通道数。
            c2 (int): 输出通道数。
            n (int): Bottleneck 模块的数量。
            shortcut (bool): 是否使用 shortcut 连接（跳跃连接）。
            g (int): 卷积操作中的分组数。
            e (float): 扩展比例。
            k (int): 卷积核的大小。
        """
        # 调用父类C3的初始化方法
        super().__init__(c1, c2, n, shortcut, g, e)
        c_ = int(c2 * e)  # 隐藏通道数
        # 使用自定义大小的卷积核初始化Bottleneck模块
        # self.m = nn.Sequential(*(RepBottleneck(c_, c_, shortcut, g, k=(k, k), e
        # for _ in range(n)))
        # 将每个Bottleneck模块的卷积核大小设置为(k, k)
        self.m = nn.Sequential(*(Bottleneck(c_, c_, shortcut, g, k=(k, k), e=1.0)
                                 for _ in range(n)))
```

炮哥带你学出品，盗版必究

Yolov11中的C3k2模块



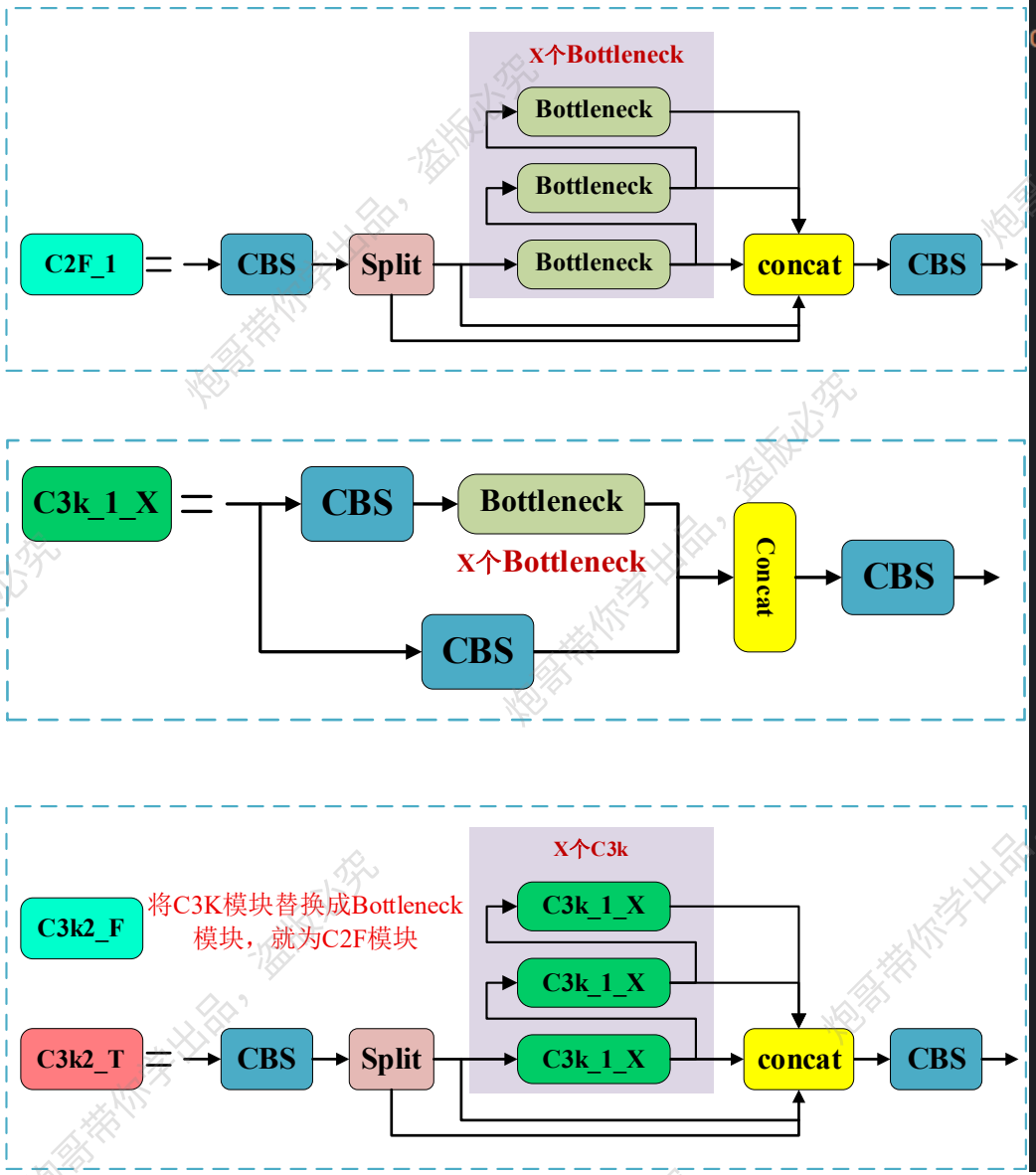





```python
class C2f(nn.Module):
    def __init__(self, c1: int, c2: int, n: int = 1, shortcut: bool = False,
                 g: int = 1, e: float = 0.5):
        """
        参数:
            c1 (int): 输入通道数。
            c2 (int): 输出通道数。
            n (int): Bottleneck模块的数量。
            shortcut (bool): 是否使用shortcut连接（跳跃连接）。
            g (int): 卷积操作中的分组数。
            e (float): 扩展比例，决定隐藏通道的数量。
        """
        # 计算隐藏通道数
        super().__init__()
        self.c = int(c2 * e)  # 隐藏通道数
        # 第一层卷积，1x1卷积
        self.cv1 = Conv(c1, 2 * self.c, k: 1, s: 1)
        # 第二层卷积，1x1卷积，最终输出通道数为c2
        self.cv2 = Conv((2 + n) * self.c, c2, k: 1)  # 可选的激活函数：FReLU(c2)
        # 使用多个Bottleneck模块
        self.m = nn.ModuleList(Bottleneck(self.c, self.c, shortcut, g,
                               k=((3, 3), (3, 3)),e=1.0) for _ in range(n))
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # 将输入通过cv1卷积后，按通道维度拆分为2部分
        y = list(self.cv1(x).chunk(2, 1))
        # 将拆分后的部分与Bottleneck模块的输出进行连接
        y.extend(m(y[-1]) for m in self.m)
        # 将最终的输出通过cv2卷积得到结果
        return self.cv2(torch.cat(y, dim: 1))
```

炮哥带你学出品，盗版必究

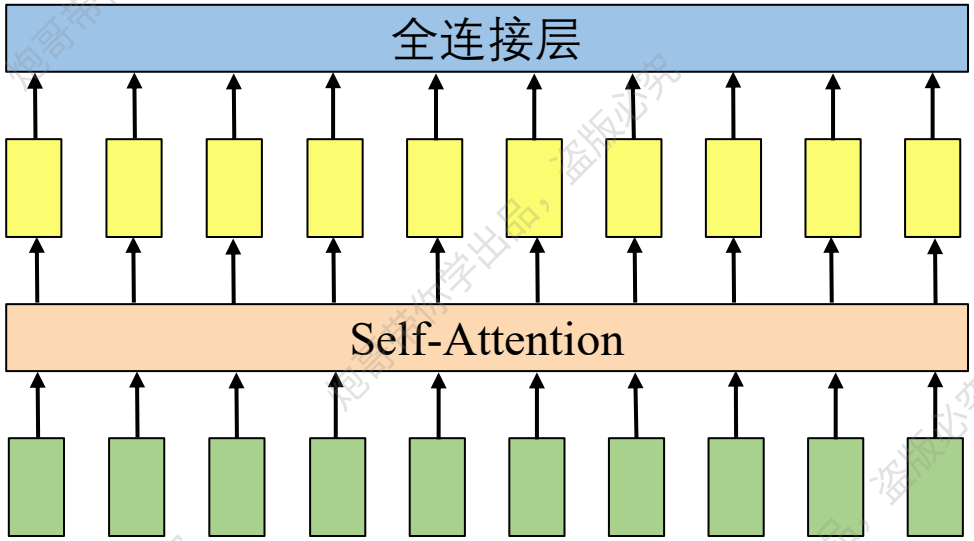
Yolov11中的C3k2模块

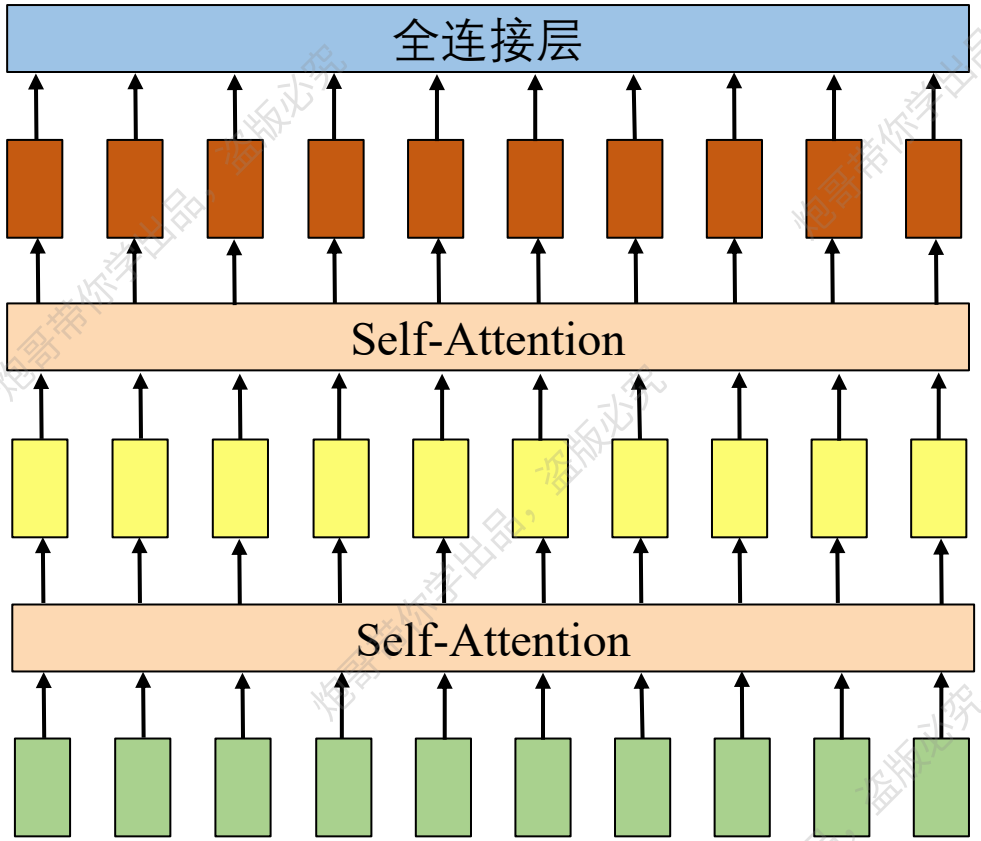



```python
class C3k2(C2f):
    def __init__(
        self, c1: int, c2: int, n: int = 1, c3k: bool = False, e: float = 0.5,
            g: int = 1, shortcut: bool = True):
        """
        参数:
            c1 (int): 输入通道数。
            c2 (int): 输出通道数。
            n (int): Bottleneck模块的数量。
            c3k (bool): 是否使用 C3k 模块。
            e (float): 扩展比例。
            g (int): 卷积操作中的分组数。
            shortcut (bool): 是否使用shortcut连接（跳跃连接）。
        """
        # 调用父类C2f的初始化方法，初始化基础层
        super().__init__(c1, c2, n, shortcut, g, e)
        # 根据c3k的选择，使用C3k模块或Bottleneck模块
        # 如果c3k为True，则使用C3k模块；否则使用Bottleneck模块
        self.m = nn.ModuleList(
            C3k(self.c, self.c, n: 2, shortcut, g) if c3k else
            Bottleneck(self.c, self.c, shortcut, g) for _ in range(n))
```

炮哥带你学出品，盗版必究

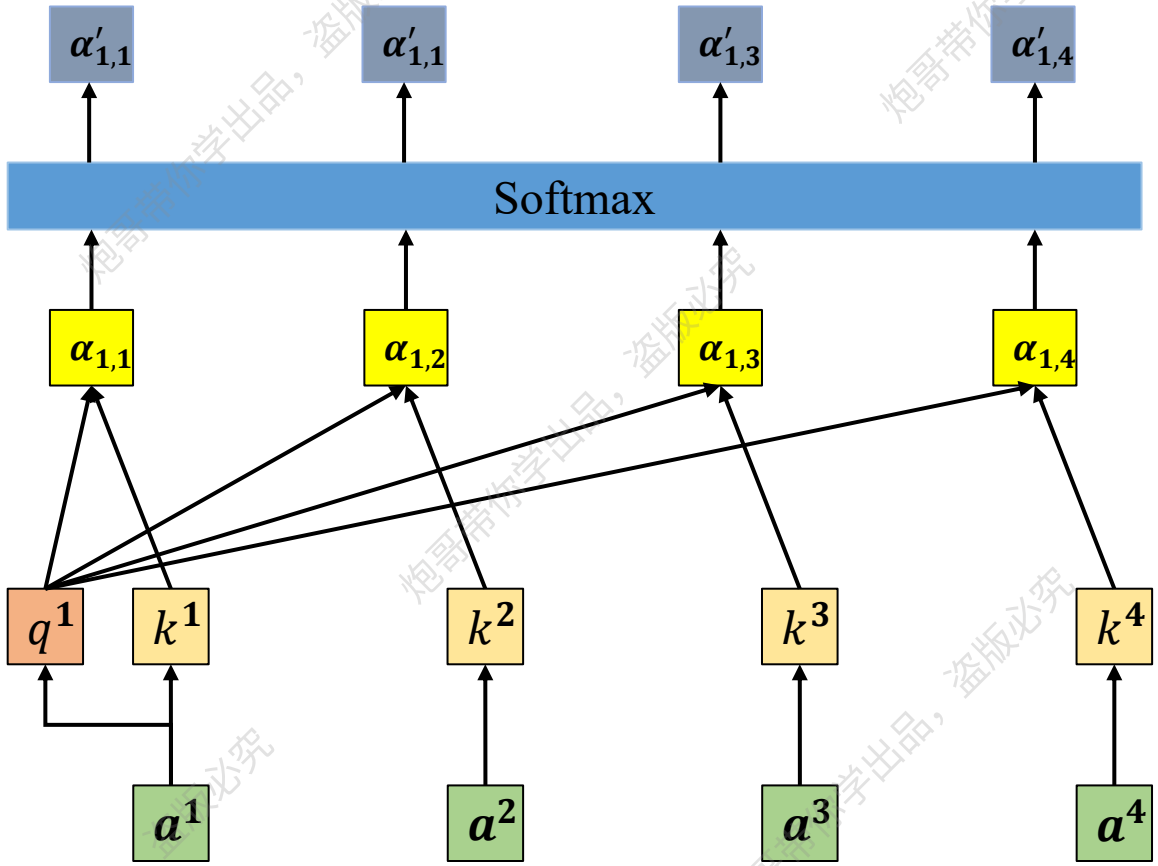
自注意力机制（Self-Attention）

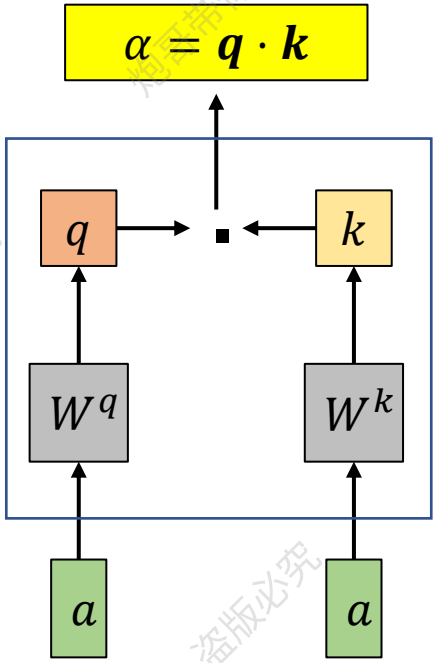




炮哥带你学出品，盗版必究

自注意力机制（Self-Attention）



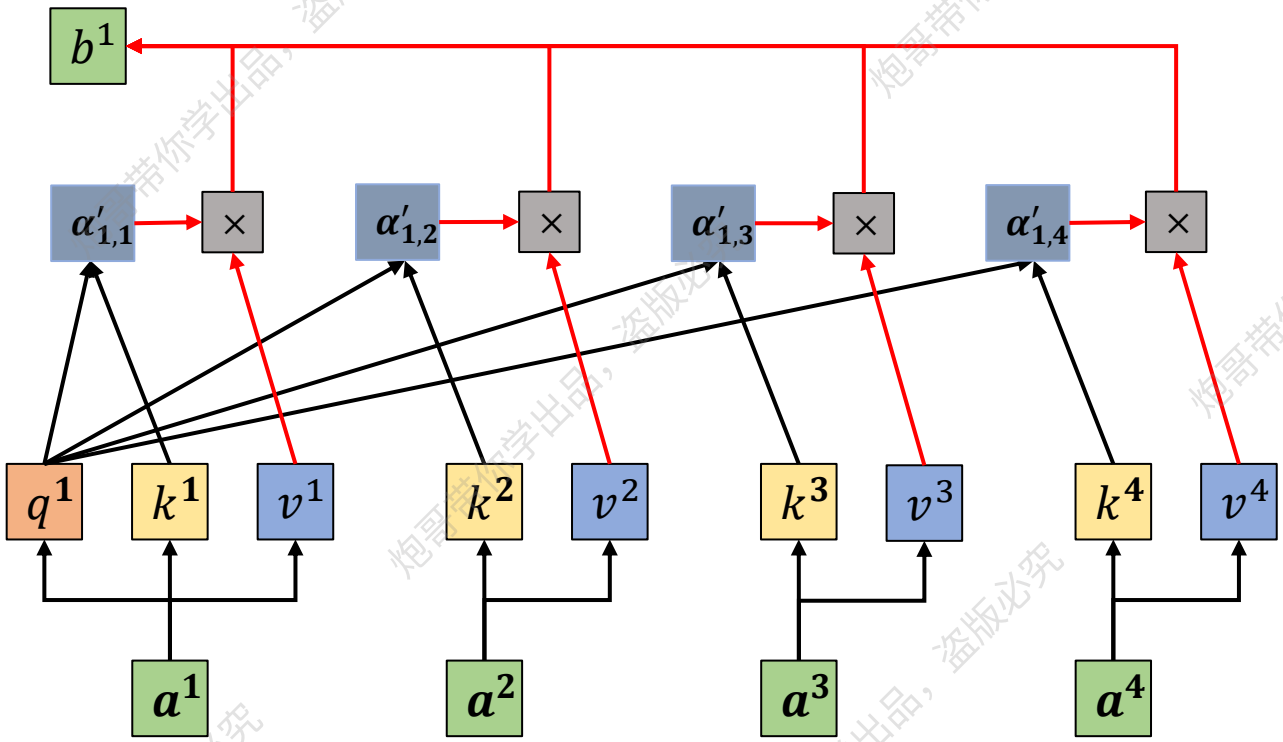


计算过程：

$$\boldsymbol{q}^1 = W^q \boldsymbol{a}^1$$
$$\boldsymbol{k}^1 = W^k \boldsymbol{a}^1$$
$$\boldsymbol{k}^2 = W^k \boldsymbol{a}^2$$
$$\boldsymbol{k}^3 = W^k \boldsymbol{a}^3$$
$$\boldsymbol{k}^4 = W^k \boldsymbol{a}^4$$

炮哥带你学出品，盗版必究

自注意力机制（Self-Attention）



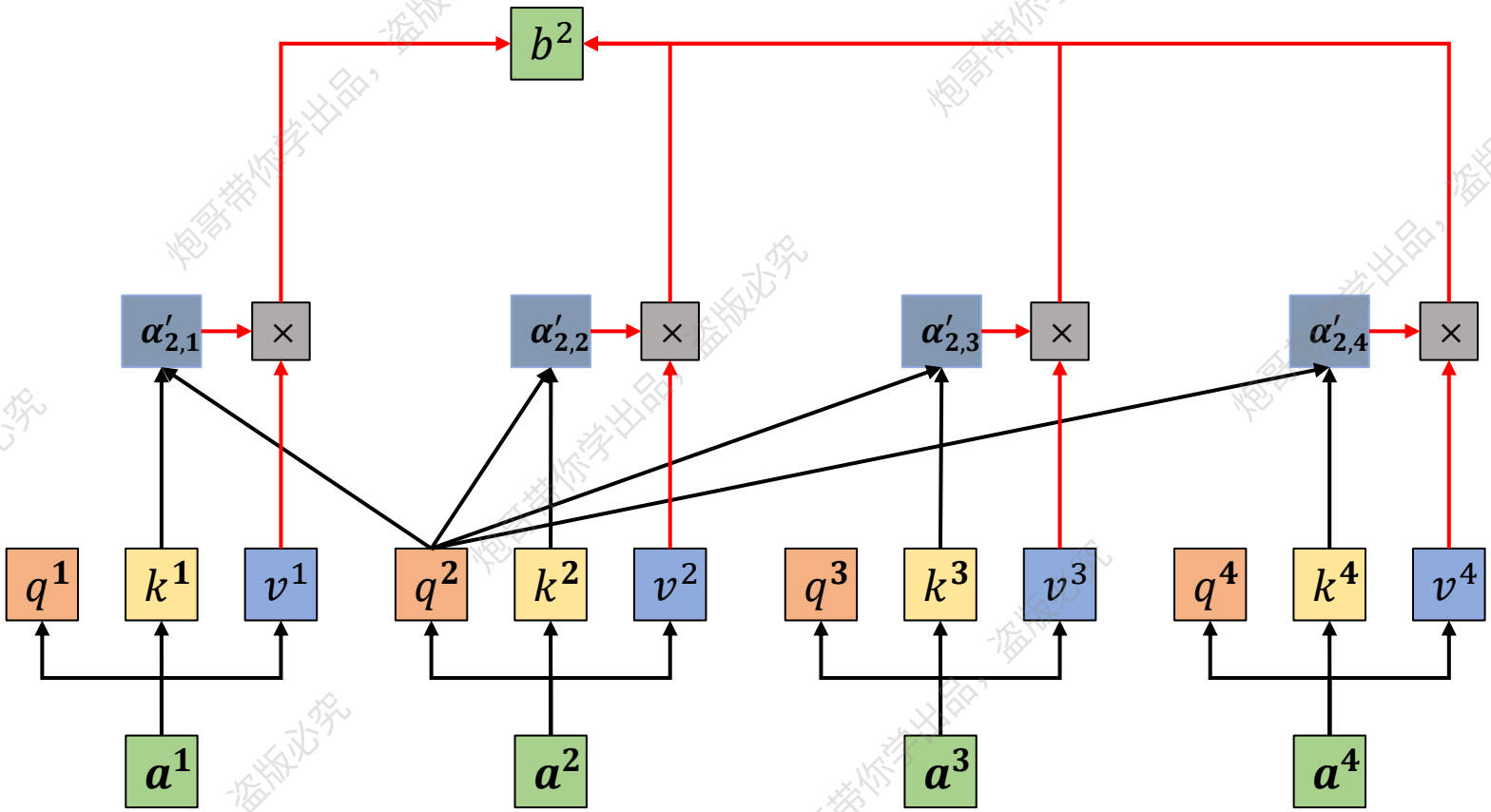
计算过程：

$$\boldsymbol{v}^1 = W^v \boldsymbol{a}^1$$
$$\boldsymbol{v}^2 = W^v \boldsymbol{a}^2$$
$$\boldsymbol{v}^3 = W^v \boldsymbol{a}^3$$
$$\boldsymbol{v}^4 = W^v \boldsymbol{a}^4$$

$$b^1 = \sum_i \alpha'_{1,i} \boldsymbol{v}^i$$

炮哥带你学出品，盗版必究

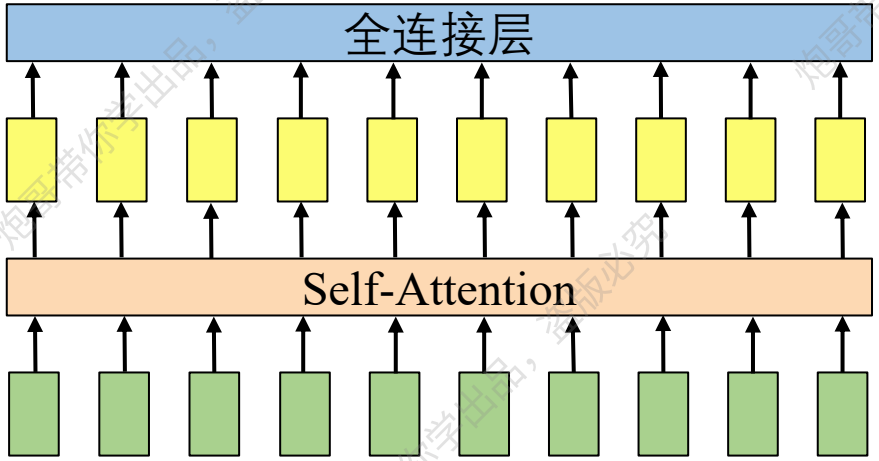
自注意力机制（Self-Attention）



计算过程：

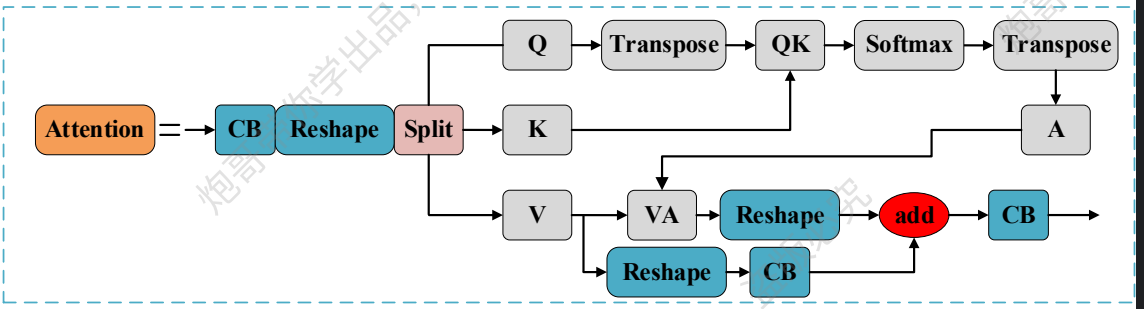
$$\boldsymbol{v}^1 = W^v \boldsymbol{a}^1$$
$$\boldsymbol{v}^2 = W^v \boldsymbol{a}^2$$
$$\boldsymbol{v}^3 = W^v \boldsymbol{a}^3$$
$$\boldsymbol{v}^4 = W^v \boldsymbol{a}^4$$

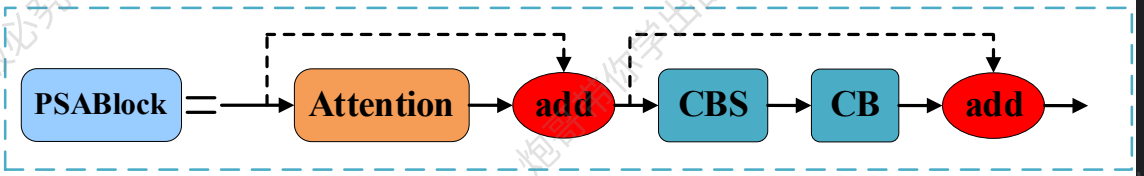
$$b^2 = \sum_i \alpha'_{2,i} \boldsymbol{v}^i$$

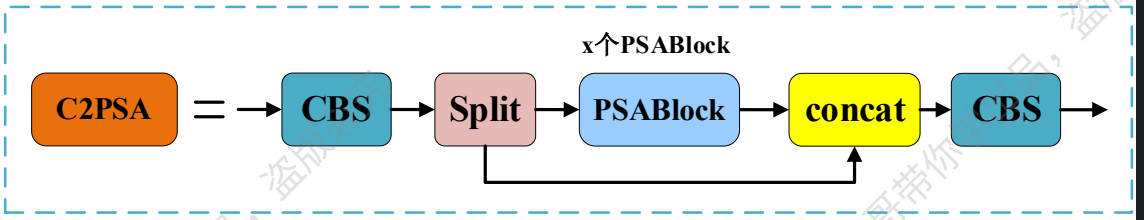


炮哥带你学出品，盗版必究

Yolov11目标检测算法的背景



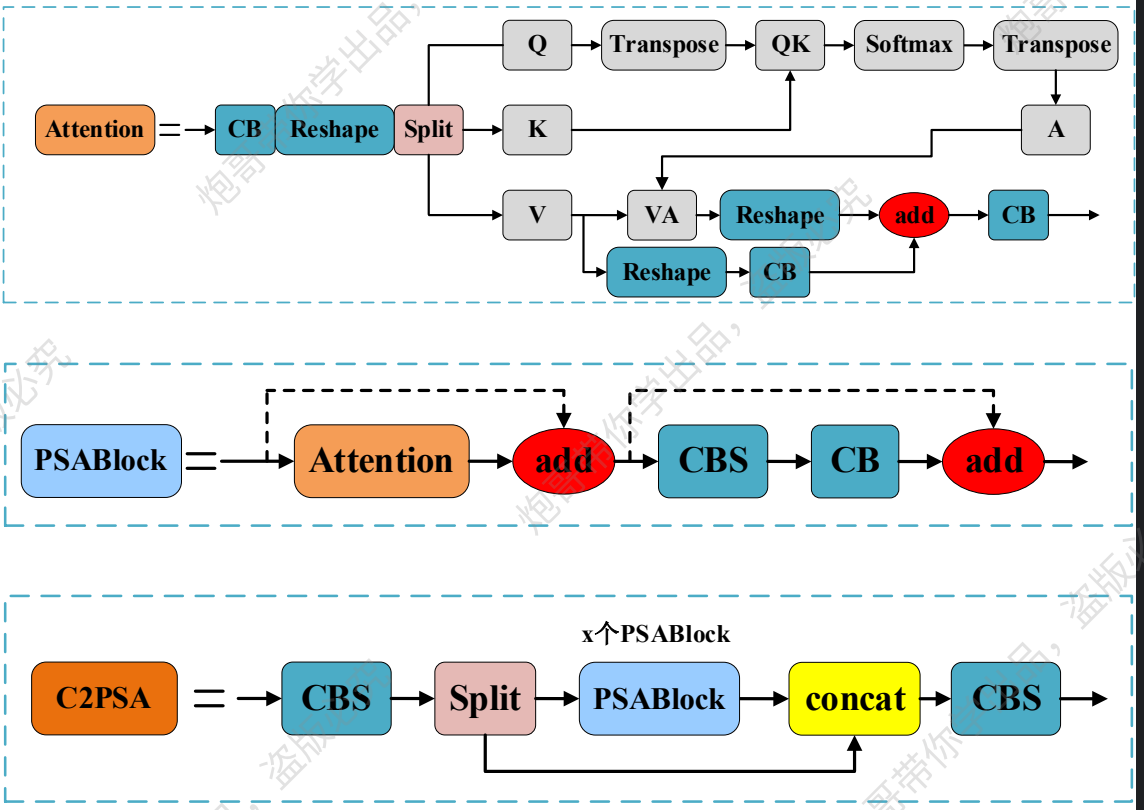





```python
class Attention(nn.Module):
    def __init__(self, dim: int, num_heads: int = 8, attn_ratio: float = 0.5):
        super().__init__()
        self.num_heads = num_heads    # 注意力头数
        self.head_dim = dim // num_heads  # 每个头的维度
        self.key_dim = int(self.head_dim * attn_ratio)  # key 的通道数
        self.scale = self.key_dim**-0.5  # 分数缩放（避免内积随维度增大而过大）
        nh_kd = self.key_dim * num_heads  # 所有头的 key 总通道数
        h = dim + nh_kd * 2  # Q/K/V 通道总数 = V(dim) + Q(nh_kd) + K(nh_kd)
        self.qkv = Conv(dim, h, k: 1, act=False)   # 1x1 卷积一次性产生 Q/K/V
        self.proj = Conv(dim, dim, k: 1, act=False)  # 1x1 卷积做输出投影
        self.pe = Conv(dim, dim, k: 3, s: 1, g=dim, act=False)# 3x3 深度可分离卷积作为位置编码（
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, C, H, W = x.shape
        N = H * W  # token 数：空间展平后的位置数
        qkv = self.qkv(x)  # 生成拼接的 Q/K/V，形状：[B, h, H, W]，其中 h = dim + 2*num_heads*k

        # 将通道展开为 (num_heads, key_dim/key_dim/head_dim) 三段，并把空间展平为 N
        q, k, v = qkv.view(B, self.num_heads, self.key_dim * 2 + self.head_dim, N).split(
            [self.key_dim, self.key_dim, self.head_dim], dim=2
        )
        # 注意力分数：q^T k → [B, num_heads, N, N]，再做 softmax
        attn = (q.transpose(-2, -1) @ k) * self.scale
        attn = attn.softmax(dim=-1)   # 对每个 query 位置在所有 key 上归一化
        # 将注意力作用到 v 上（这里等价于 V @ Attn^T）
        # 上式右侧第二项为位置编码（对 v 做 depthwise 3x3 卷积后相加，增强空间位置信息）
        x = (v @ attn.transpose(-2, -1)).view(B, C, H, W) + self.pe(v.reshape(B, C, H, W))
        x = self.proj(x)  # 输出投影
        return x
```

炮哥带你学出品，盗版必究

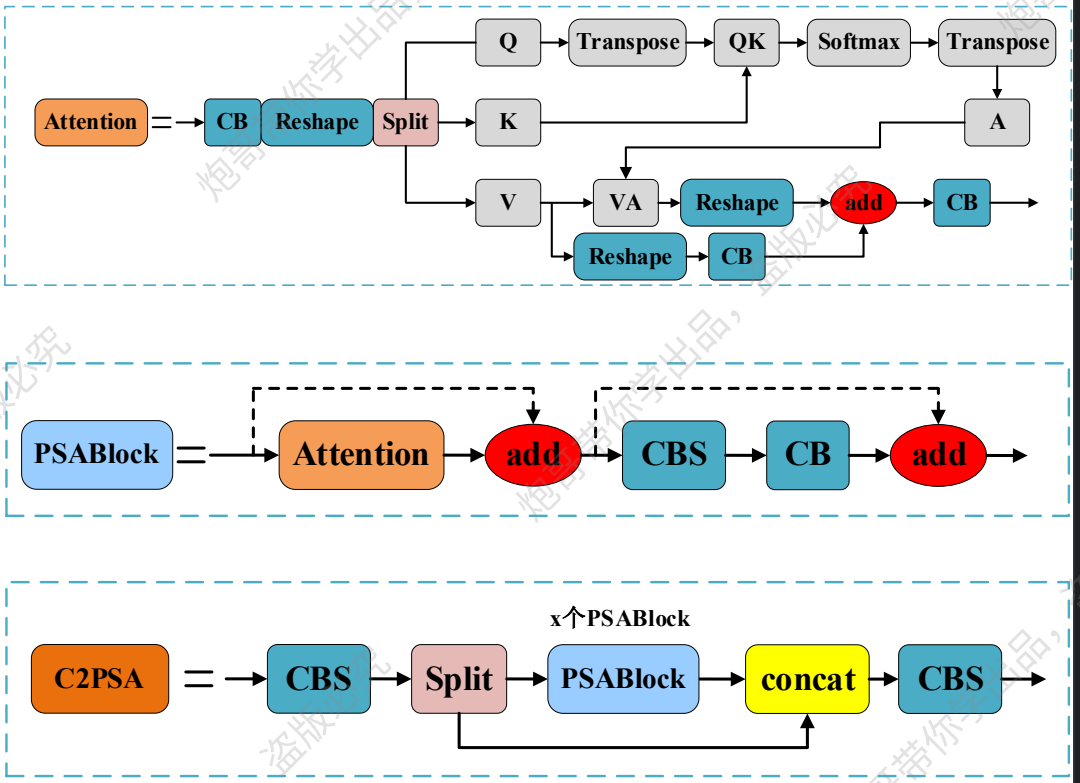
Yolov11中的Attention模块




```python
class Attention(nn.Module):
    def __init__(self, dim: int, num_heads: int = 8, attn_ratio: float = 0.5):
        super().__init__()
        self.num_heads = num_heads    # 注意力头数
        self.head_dim = dim // num_heads  # 每个头的维度
        self.key_dim = int(self.head_dim * attn_ratio)  # key 的通道数
        self.scale = self.key_dim**-0.5  # 分数缩放（避免内积随维度增大而过大）
        nh_kd = self.key_dim * num_heads  # 所有头的 key 总通道数
        h = dim + nh_kd * 2  # Q/K/V 通道总数 = V(dim) + Q(nh_kd) + K(nh_kd)
        self.qkv = Conv(dim, h, k: 1, act=False)   # 1x1 卷积一次性产生 Q/K/V
        self.proj = Conv(dim, dim, k: 1, act=False)  # 1x1 卷积做输出投影
        self.pe = Conv(dim, dim, k: 3, s: 1, g=dim, act=False)# 3x3 深度可分离卷积作为位置编码（
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, C, H, W = x.shape
        N = H * W  # token 数：空间展平后的位置数
        qkv = self.qkv(x)  # 生成拼接的 Q/K/V，形状：[B, h, H, W]，其中 h = dim + 2*num_heads*k

        # 将通道展开为 (num_heads, key_dim/key_dim/head_dim) 三段，并把空间展平为 N
        q, k, v = qkv.view(B, self.num_heads, self.key_dim * 2 + self.head_dim, N).split(
            [self.key_dim, self.key_dim, self.head_dim], dim=2
        )
        # 注意力分数：q^T k → [B, num_heads, N, N]，再做 softmax
        attn = (q.transpose(-2, -1) @ k) * self.scale
        attn = attn.softmax(dim=-1)   # 对每个 query 位置在所有 key 上归一化
        # 将注意力作用到 v 上（这里等价于 V @ Attn^T）
        # 上式右侧第二项为位置编码（对 v 做 depthwise 3x3 卷积后相加，增强空间位置信息）
        x = (v @ attn.transpose(-2, -1)).view(B, C, H, W) + self.pe(v.reshape(B, C, H, W))
        x = self.proj(x)  # 输出投影
        return x
```

炮哥带你学出品，盗版必究

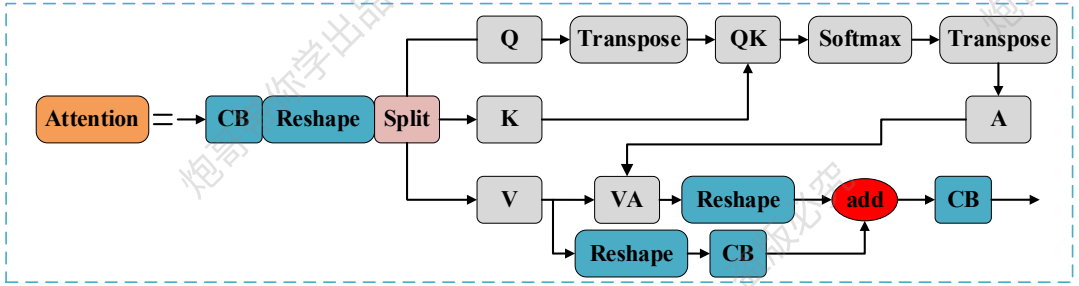
Yolov11中的C2PSA模块

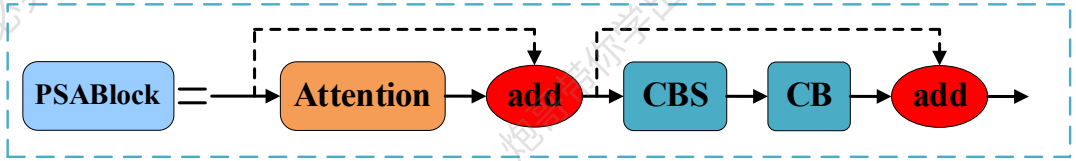


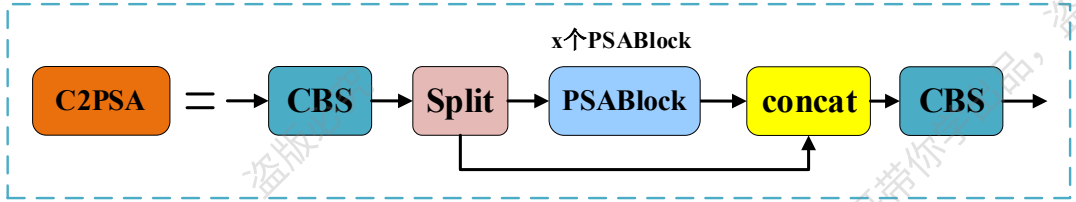

```python
class PSABlock(nn.Module):  2 usages
    def __init__(self, c: int, attn_ratio: float = 0.5,
                 num_heads: int = 4, shortcut: bool = True) -> None:
        """
        Initialize the PSABlock.

        参数:
            c (int): 输入和输出的通道数。
            attn_ratio (float): 注意力机制中键的维度比例。
            num_heads (int): 多头注意力机制中的头数。
            shortcut (bool): 是否使用shortcut连接。
        """
        super().__init__()

        # 多头注意力子层（位置/空间敏感），输入输出通道均为 c，头数为 num_heads
        self.attn = Attention(c, attn_ratio=attn_ratio, num_heads=num_heads)
        # 前馈网络：1x1 卷积升维到 2c，再用 1x1 卷积降回 c（第二层 act=False 表示无激活）
        self.ffn = nn.Sequential(Conv(c, c * 2, k: 1), Conv(c * 2, c, k: 1, act=False))
        # 是否启用残差（shortcut）连接：True 时执行 x + 子层(x)
        self.add = shortcut

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # 子层1：注意力；若开启残差则 x = x + Attn(x)，否则直接 Attn(x)
        x = x + self.attn(x) if self.add else self.attn(x)
        # 子层2：前馈网络；同样的残差策略
        x = x + self.ffn(x) if self.add else self.ffn(x)
        return x
```

炮哥带你学出品，盗版必究

Yolov11中的C2PSA模块





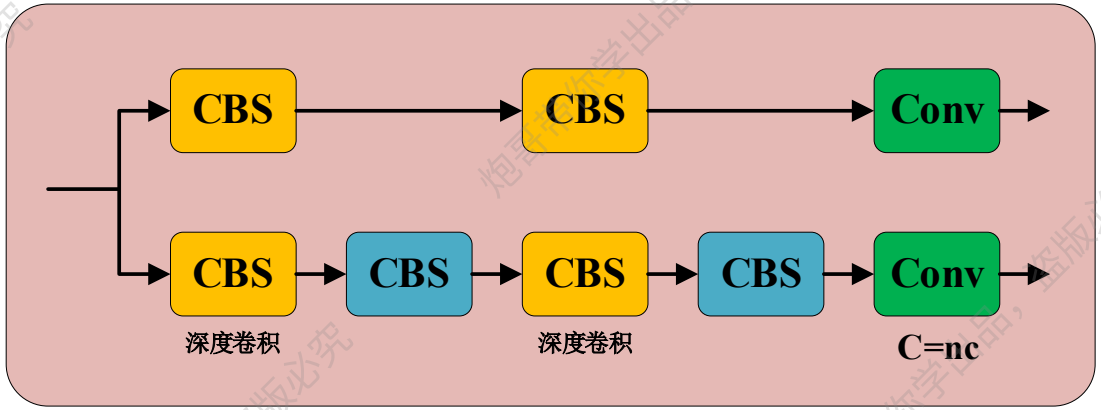



```python
class C2PSA(nn.Module):
    """
    该模块本质上与PSA模块相同，但进行了重构，以允许堆叠更多的PSABlock模块。
    """
    def __init__(self, c1: int, c2: int, n: int = 1, e: float = 0.5):
        """
        参数:
            c1 (int): 输入通道数。
            c2 (int): 输出通道数。
            n (int): PSABlock模块的数量。
            e (float): 扩展比例。
        """
        super().__init__()
        assert c1 == c2  # 输入和输出通道数应当相等
        self.c = int(c1 * e)  # 计算隐藏通道数
        self.cv1 = Conv(c1, 2 * self.c, k: 1, s: 1)  # 第一个1x1卷积，扩展通道数
        self.cv2 = Conv(2 * self.c, c1, k: 1)  # 第二个1x1卷积，恢复通道数

        # 使用多个PSABlock模块，堆叠n个PSABlock模块
        self.m = nn.Sequential(*(PSABlock(self.c, attn_ratio=0.5,
                                          num_heads=self.c // 64) for _ in range(n)))
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # 使用cv1卷积层将输入张量分成两个部分
        a, b = self.cv1(x).split((self.c, self.c), dim=1)
        # 将b部分通过PSABlock模块进行处理
        b = self.m(b)
        # 将a和处理后的b拼接在一起，并通过cv2卷积层得到输出
        return self.cv2(torch.cat( tensors: (a, b),  dim: 1))
```

炮哥带你学出品，盗版必究

Yolov11中的检测头

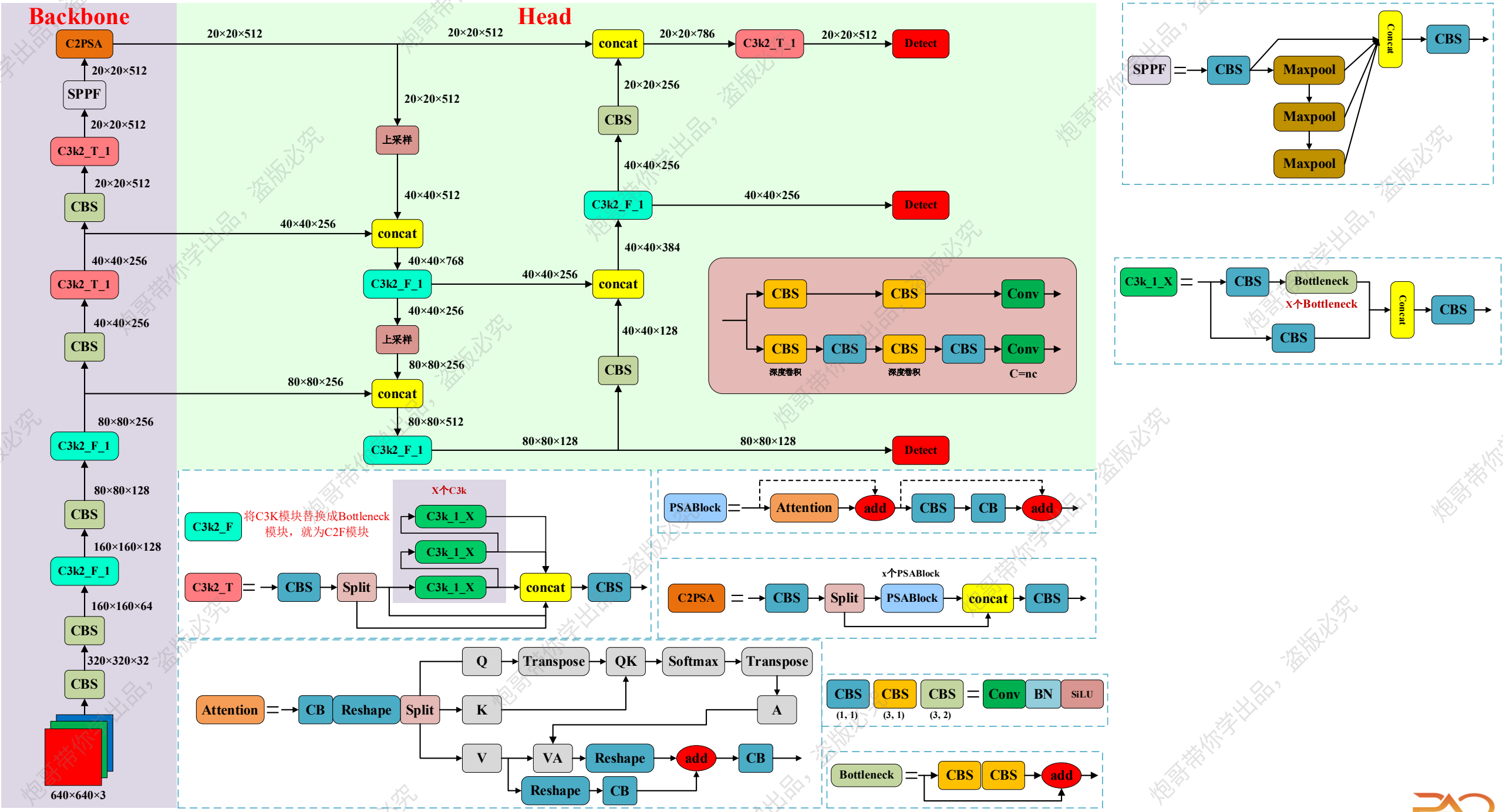
这里的检测头完全继承了yolov10的检测头，所以这里我不得不发出一个感叹，yolo系列之所以发展到现在又快又准，都是不同开发者共同的努力的结果。




```python
class DWConv(Conv):  16 usages
    def __init__(self, c1, c2, k=1, s=1, d=1, act=True):
        """
        参数:
        #   c1: 输入通道数
        #   c2: 输出通道数
        #   k: 卷积核大小
        #   s: 步幅
        #   d: 空洞率 (dilation)
        """
        # groups = math.gcd(c1, c2) (c1 与 c2 的最大公约数)
        super().__init__(c1, c2, k, s, g=math.gcd( *integers: c1, c2), d=d, act=act)
```

```python
self.cv2 = nn.ModuleList(
    nn.Sequential(Conv(x, c2, k: 3), Conv(c2, c2, k: 3),
                  nn.Conv2d(c2, 4 * self.reg_max, kernel_size: 1)) for x in ch
)
self.cv3 = (
    nn.ModuleList(nn.Sequential(Conv(x, c3, k: 3), Conv(c3, c3, k: 3),
                                nn.Conv2d(c3, self.nc, kernel_size: 1)) for x in ch)
    if self.legacy
    else nn.ModuleList(
        nn.Sequential(
            nn.Sequential(DWConv(x, x, k: 3), Conv(x, c3, k: 1)),
            nn.Sequential(DWConv(c3, c3, k: 3), Conv(c3, c3, k: 1)),
            nn.Conv2d(c3, self.nc, kernel_size: 1),
        )
        for x in ch
```

炮哥带你学出品，盗版必究


Backbone
Head
C2PSA
SPPF
C3k2_T_1
CBS
C3k2_T_1
CBS
C3k2_F_1
CBS
C3k2_F_1
CBS
C3k2_F_1
640×640×3
20×20×512
20×20×512
20×20×512
20×20×512
40×40×256
40×40×256
40×40×256
80×80×256
80×80×256
80×80×128
160×160×128
160×160×64
320×320×32
上采样
40×40×512
concat
40×40×768
C3k2_F_1
40×40×256
上采样
80×80×256
concat
80×80×512
C3k2_F_1
concat
20×20×786
C3k2_T_1
20×20×512
Detect
20×20×256
CBS
40×40×256
C3k2_F_1
40×40×256
Detect
40×40×384
concat
40×40×128
CBS
80×80×128
80×80×128
Detect
CBS
CBS
Conv
CBS
CBS
CBS
CBS
Conv
深度卷积
深度卷积
C=nc
SPPF
CBS
Maxpool
Maxpool
Maxpool
Concat
CBS
C3k_1_X
CBS
Bottleneck
x个Bottleneck
CBS
Concat
CBS
C3k2_F
将C3K模块替换成Bottleneck模块，就为C2F模块
x个C3k
C3k_1_X
C3k_1_X
C3k_1_X
C3k2_T
CBS
Split
concat
CBS
PSABlock
Attention
add
CBS
CB
add
C2PSA
CBS
Split
x个PSABlock
PSABlock
concat
CBS
Attention
CB
Reshape
Split
Q
K
V
Transpose
QK
Softmax
Transpose
A
VA
Reshape
add
CB
Reshape
CB
CBS
CBS
CBS
(1, 1)
(3, 1)
(3, 2)
Conv
BN
SiLU
Bottleneck
CBS
CBS
add